Inheritance and Polymorphism

Store an ArrayList in ascending order

```
import java.util.ArrayList;
import java.util.Scanner;

public static void main(String [] args) {
     ArrayList<Integer> numbers = new ArrayList<Integer>();
     System.out.println("Enter numbers one after another. Once done enter -1");

    \\  Keep scanning from user until user enters -1
     \\ Store the numbers in ascending order as the user enters
              \\ have a loop to check if the number where the number should be and insert it in
                 that index
     \\ Print out the final array.

}
```

SORTING HAS TO HAPPEN IN REAL TIME as user inputs. DO NOT USE COLLECTIONS.SORT()

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        int flag = 0;
        Scanner sc = new Scanner(System.in);
        ArrayList<Integer> numbers = new ArrayList<Integer>();
        Integer x = sc.nextInt();           // scan the very first number
        while (!x.equals(-1)) {             // if the first number is -1, while loop does not get execute
            if (numbers.size() == 0){       // this will get executed just once
                numbers.add(x);             // very first element gets added to ArrayList
                x=sc.nextInt();             // second element gets scanned here
                continue;                   // remaining of the while loop is not executed
            }
            for (int j = 0; j < numbers.size(); j++) {  // for loop to iterate through ArrayList
                if (x < numbers.get(j)) {
                    numbers.add(j,x);                   // TRUE if new element is < existing element
                    flag = 1;
                    break;                              // will break as long as at least one element ...
                }                                       //   ..... is larger than the new element
            }
            if (flag == 0){
                numbers.add(x);                         // will execute if new element is largest
            }
            x=sc.nextInt();                             // scan new element
            flag = 0;                                   // should reset the flag for next iteration
        }
        System.out.println(numbers);
    }
}
```

Java Inheritance (Subclass and Superclass)

In Java, it is possible to inherit attributes and methods from one class to another. We group the "inheritance concept" into two categories:

- **subclass** (child) - the class that inherits from another class
- **superclass** (parent) - the class being inherited from

To inherit from a class, use the `extends` keyword.

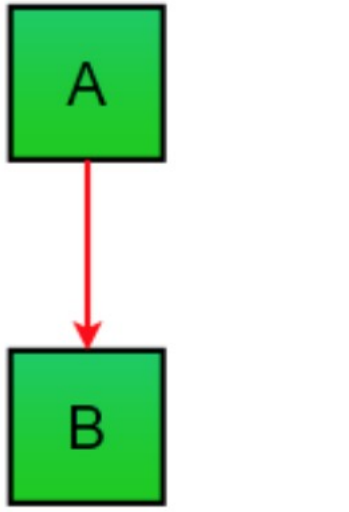
Subclasses inherit things it knows (variables) and things is does (methods)

If you don't want other classes to inherit from a class, use the `final` keyword

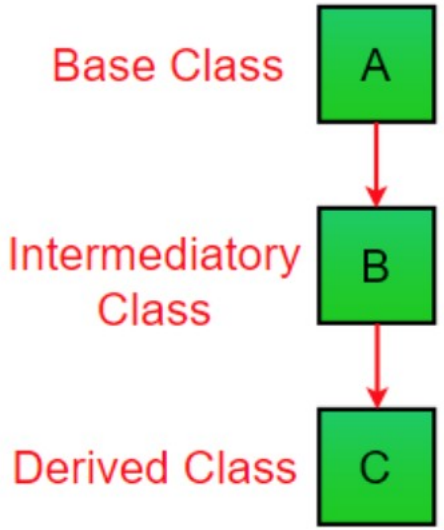
Access specifiers for Member Access

Specifier	Description
private	Accessible by self.
protected	Accessible by self, derived classes, and other classes in the same package.
public	Accessible by self, derived classes, and everyone else.
no specifier	Accessible by self and other classes in the same package.

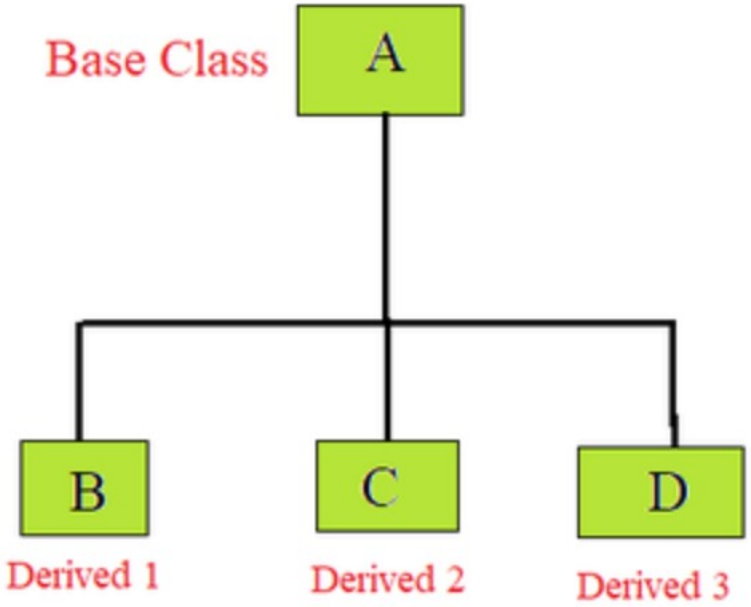
Types of Inheritance



Single Inheritance

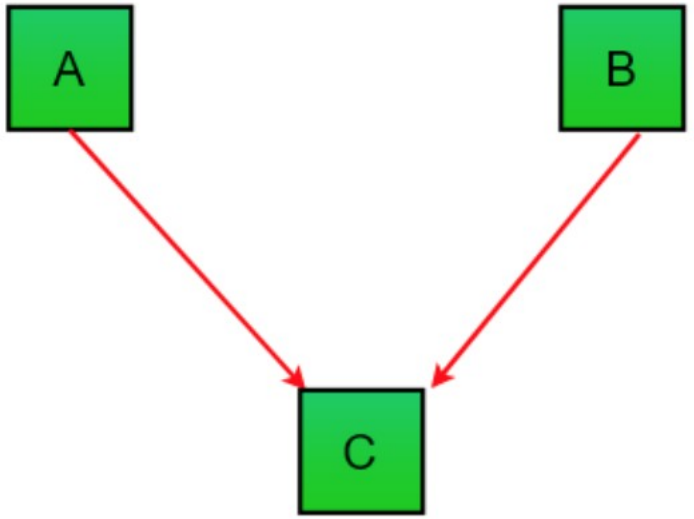


Multilevel Inheritance



Hierarchical Inheritance

Multiple Inheritance

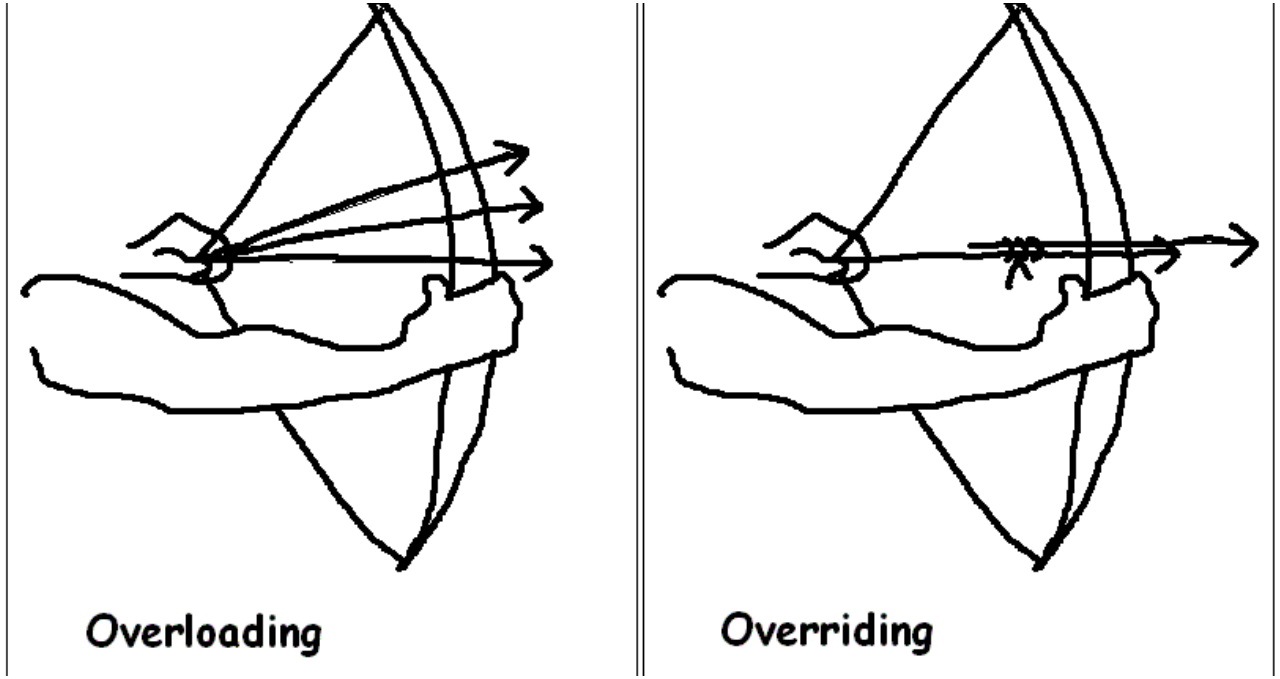


Java does not support multiple inheritance in classes. However, multiple inheritance can be achieved using interfaces (next chapter)

Inheritance Facts – JAVA

- **Default superclass**: Except Object class, which has no superclass, every class has one and only one direct superclass (single inheritance). In the absence of any other explicit superclass, every class is implicitly a subclass of the Object class.
- **Superclass can only be one:** A superclass can have any number of subclasses. But a subclass can have only **one** superclass. This is because Java does not support multiple inheritances with classes. Although with interfaces, multiple inheritances are supported by java.
- **Inheriting Constructors:** A subclass inherits all the members (fields, methods, and nested classes) from its superclass. Constructors are not members, so they are not inherited by subclasses, but the constructor of the superclass can be invoked from the subclass.
- **Private member inheritance:** A subclass does not inherit the private members of its parent class. However, if the superclass has public or protected methods(like getters and setters) for accessing its private fields, these can also be used by the subclass.

Overloading vs Overriding



Overloading: methods with the same name but have different parameter types.

Overriding: a subclass method takes precedence over a superclass method with the same name and parameter types but probably with different implementations.

Overriding

The ***@Override*** annotation is placed above a method that overrides a base class method so the compiler verifies that an identical base class method exists.

An ***annotation*** is an optional command beginning with the "@" symbol that can provide the compiler with information that helps the compiler detect errors better.

An overriding method can call the overridden method by using the super keyword.
Ex: `super.getDescription()`.

The ***super*** keyword is a reference variable used to call the parent class's methods or constructors.

Object Class

The built-in ***Object class*** serves as the base class for all other classes and does not have a base class.

- The ***toString()*** method returns a String representation of the Object. By default, toString() returns a String containing the object's class name followed by the object's hexadecimal address in memory. Ex: java.lang.Object@372f7a8d.

- The ***equals(otherObject)*** method compares an Object to otherObject and returns true if both variables reference the same object. Otherwise, equals() returns false. By default, equals() tests the equality of the two Object references, not the equality of the Objects' contents.

Polymorphism

Polymorphism refers to determining which program behavior to execute depending on data types. Method overloading is a form of ***compile-time polymorphism*** wherein the compiler determines which of several identically-named methods to call based on the method's arguments.

Another form is ***runtime polymorphism*** wherein the compiler cannot make the determination but instead the determination is made while the program is running.

For objects → casting is done implicitly in JAVA

Compile-time polymorphism

```
public Inventory(){
    FruitType = "some fruit";
    FruitQuantity = -1;
    FruitPrice = -1;
}

public Inventory(String Fruitname){
    FruitType = Fruitname;
    FruitQuantity = -1;
    FruitPrice = -1;
}

public Inventory(String Fruitname, int NewItems){
    FruitType = Fruitname;
    FruitQuantity = NewItems;
    FruitPrice = -1;
}
```

Constructor overloading and method overloading
Are typical examples of ***compile-time polymorphism***

Runtime polymorphism

```java
import java.util.ArrayList;

public class ItemInventory {
   public static void main(String[] args) {
      GenericItem genericItem1;
      ProduceItem produceItem1;
      ArrayList<GenericItem> inventoryList = new ArrayList<GenericItem>(); // Collection of "Items"
      int i;                                                               // Loop index

      genericItem1 = new GenericItem();
      genericItem1.setName("Smith Cereal");
      genericItem1.setQuantity(9);

      produceItem1 = new ProduceItem();
      produceItem1.setName("Apple");
      produceItem1.setQuantity(40);
      produceItem1.setExpiration("May 5, 2012");

      genericItem1.printItem();
      produceItem1.printItem();

      // More common: Collection (e.g., ArrayList) of objs
      // Polymorphism -- Correct  printItem()  called
      inventoryList.add(genericItem1);
      inventoryList.add(produceItem1);
      System.out.println("\nInventory: ");
      for (i = 0; i < inventoryList.size(); ++i) {
         inventoryList.get(i).printItem(); // Calls correct printItem()
      }
   }
}
```

Printing objects from ArrayList Is an example of runtime polymorphism

ArrayList of Objects

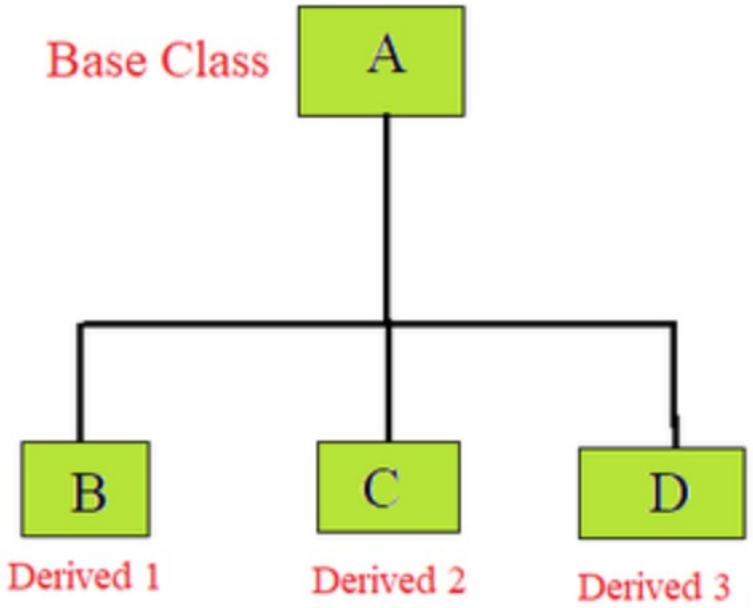
Because all classes are derived from the Object class, programmers can take advantage of runtime polymorphism in order to create a collection (e.g., ArrayList) of objects of various class types and perform operations on the elements.

```
public static void main(String[] args) {
    ArrayList<Object> objList = new ArrayList<Object>();

    // Add new instances of various classes to objList
    objList.add(new Object());
    objList.add(12);
    objList.add(3.14);
    objList.add(new String("Hello!"));
    objList.add(new Business("ACME", "5 Main St"));

    // Print list of Objects
    printArrayList(objList);
}
```

Is-a versus has-a relationship




```
public class PersonInfo {
   public String firstName;
   public String birthdate;

   ...
}

public class ChildInfo extends PersonInfo {
   public String schoolName;

   ...
}

public class MotherInfo extends PersonInfo {
   public String spousename;
   public ArrayList<ChildInfo> childrenData;
  ...
}
```

Mother is a Person
Child is a person

Is-a versus has-a relationship

```
public class ChildInfo {
   public String firstName;
   public String birthDate;
   public String schoolName;

   ...
}

public class MotherInfo {
   public String firstName;
   public String birthDate;
   public String spouseName;
   public ArrayList<ChildInfo> childrenData;

   ...
}
```

The 'has-a' relationship. A MotherInfo object 'has a' String object and 'has a' ArrayList of ChildInfo objects, but no inheritance is involved.

This is an example of composition. Composition should not be confused with inheritance.